

Data Cleaning

```
In [1]: import os

import pandas as pd

ROOT_PATH = os.path.dirname(os.getcwd())
DATA_FOLDER_PATH = os.path.join(ROOT_PATH, 'data', 'raw')
```

Robust Mapping of Text Fields

The raw files spell the same value in different ways. Older files carry broken encodings (RONDnia , where the accented letter was lost), and newer ones use the proper accents (Rondônia). A plain dictionary lookup would need every variant, and any value missing from the dictionary would silently become NaN .

`map_strict` solves both problems:

- **Matching key.** Each value is lower-cased and every non-ASCII character (an accent or the `?` replacement character) is replaced by `?`. `Rondônia` and `RONDnia` both become `rond?nia`, so one dictionary entry covers both spellings.
- **Loud failure.** If a value has no entry, the notebook stops and lists the missing values and the dictionary to update. New data can never be dropped silently.

```
In [2]: def text_key(value) -> str:
        """Lower-case key where every non-ASCII character (accent or '?') becomes '?'.
        return ''.join(ch if ch.isascii() else '?' for ch in str(value).strip().lower())

def build_lookup(mapping: dict, mapping_name: str) -> dict:
    lookup = {}
    for raw, target in mapping.items():
        key = text_key(raw)
        if lookup.get(key, target) != target:
            raise ValueError(
                f'{mapping_name}: {raw!r} and another entry share the key {key!r} '
                f'but map to different values ({lookup[key]!r} vs {target!r})'
            )
        lookup[key] = target
    return lookup

def map_strict(values: pd.Series, mapping: dict, mapping_name: str) -> pd.Series:
    """Map values with an accent/encoding-insensitive key; fail on unknown values."""
    lookup = build_lookup(mapping, mapping_name)
    translation = {
        value: lookup.get(text_key(value)) for value in values.dropna().unique()
    }
```

```

unmapped = sorted(value for value, target in translation.items() if target is N
if unmapped:
    raise ValueError(
        f'{len(unmapped)} value(s) missing from {mapping_name}: {unmapped}. '
        f'Add them to {mapping_name} and re-run.'
    )
return values.map(translation)

```

Import

```

In [3]: df = pd.read_parquet(os.path.join(DATA_FOLDER_PATH, 'raw_arrivals.parquet'))

df.head()

```

```

Out[3]:

```

	access_route	state	origin_country	month	year	arrival_count
0	Aérea	Bahia	Afeganistão	Janeiro	2026	1.0
1	Aérea	São Paulo	Afeganistão	Janeiro	2026	4.0
2	Terrestre	Paraná	Afeganistão	Janeiro	2026	1.0
3	Terrestre	Rio Grande do Sul	Afeganistão	Janeiro	2026	1.0
4	Marítima	Rio de Janeiro	Afeganistão	Janeiro	2026	3.0

Consistency

```

In [4]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 995126 entries, 0 to 995125
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   access_route    995126 non-null object
1   state           995126 non-null object
2   origin_country  995126 non-null object
3   month           995126 non-null object
4   year            995126 non-null int64
5   arrival_count   985274 non-null float64
dtypes: float64(1), int64(1), object(4)
memory usage: 45.6+ MB

```

Checking NaNs

```

In [5]: df.isna().mean()

```

```
Out[5]: access_route      0.0000
state      0.0000
origin_country  0.0000
month      0.0000
year      0.0000
arrival_count  0.0099
dtype: float64
```

```
In [6]: df[df['arrival_count'].isna()].head()
```

```
Out[6]:
```

	access_route	state	origin_country	month	year	arrival_count
33118	Fluvial	Mato Grosso do Sul	África do Sul	janeiro	1989	NaN
33119	Fluvial	Mato Grosso do Sul	Angola	janeiro	1989	NaN
33120	Fluvial	Mato Grosso do Sul	Nigéria	janeiro	1989	NaN
33121	Fluvial	Mato Grosso do Sul	Outros países	janeiro	1989	NaN
33122	Fluvial	Mato Grosso do Sul	Costa Rica	janeiro	1989	NaN

```
In [7]: df = df.dropna(subset=['arrival_count'])
```

Cleaning

Date field

```
In [8]: df['year'] = df['year'].astype(str)
```

```
In [9]: MONTH_NAME_MAPPING = {
    'janeiro': 'Janeiro',
    'fevereiro': 'Fevereiro',
    'março': 'Março',
    'marco': 'Março',
    'marçoo': 'Março',
    'abril': 'Abril',
    'maio': 'Maio',
    'junho': 'Junho',
    'julho': 'Julho',
    'agosto': 'Agosto',
    'setembro': 'Setembro',
    'outubro': 'Outubro',
    'novembro': 'Novembro',
    'dezembro': 'Dezembro',
}

MONTH_NUM_MAPPING = {
    'janeiro': '01',
    'fevereiro': '02',
    'março': '03',
    'abril': '04',
    'maio': '05',
```

```

    'junho': '06',
    'julho': '07',
    'agosto': '08',
    'setembro': '09',
    'outubro': '10',
    'novembro': '11',
    'dezembro': '12',
}

df['month'] = map_strict(df['month'], MONTH_NAME_MAPPING, 'MONTH_NAME_MAPPING')
df['month_num'] = map_strict(df['month'], MONTH_NUM_MAPPING, 'MONTH_NUM_MAPPING')

```

```
In [10]: df['month'].unique()
```

```
Out[10]: array(['Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho', 'Julho',
               'Agosto', 'Setembro', 'Outubro', 'Novembro', 'Dezembro'],
              dtype=object)
```

```
In [11]: df['month_num'].unique()
```

```
Out[11]: array(['01', '02', '03', '04', '05', '06', '07', '08', '09', '10', '11',
               '12'], dtype=object)
```

```
In [12]: df['date'] = pd.to_datetime(
            df['year'] + '-' + df['month_num'] + '-01'
        ) + pd.offsets.MonthEnd(0)
```

```
In [13]: df['date'].isna().sum()
```

```
Out[13]: np.int64(0)
```

```
In [14]: df['year_month'] = (
            pd
            .to_datetime(df['year'] + '-' + df['month_num'])
            .dt.to_period('M')
            .dt.to_timestamp('M')
        )
```

Access Route field

```
In [15]: df['access_route'].unique()
```

```
Out[15]: array(['Aérea', 'Terrestre', 'Marítima', 'Fluvial', 'Aéreo', 'Marítimo',
               'A♦érea', 'Mar♦ítima'], dtype=object)
```

```
In [16]: ACCESS_ROUTE_MAPPING = {
            'aérea': 'Aérea',
            'aéreo': 'Aéreo',
            'a♦érea': 'Aérea',
            'terrestre': 'Terrestre',
            'fluvial': 'Fluvial',
            'marítima': 'Marítima',
            'marítimo': 'Marítimo',
            'mar♦ítima': 'Marítima',
        }
```

```
}

df['access_route'] = map_strict(
    df['access_route'], ACCESS_ROUTE_MAPPING, 'ACCESS_ROUTE_MAPPING'
)
```

```
In [17]: df['access_route'].unique()
```

```
Out[17]: array(['Aérea', 'Terrestre', 'Marítima', 'Fluvial'], dtype=object)
```

State field

```
In [18]: df['state'].unique()
```

```
Out[18]: array(['Bahia', 'São Paulo', 'Paraná', 'Rio Grande do Sul',
                'Rio de Janeiro', 'Pará', 'Ceará', 'Santa Catarina',
                'Distrito Federal', 'Amazonas', 'Rio Grande do Norte', 'Paraíba',
                'Pernambuco', 'Alagoas', 'Minas Gerais', 'Rondônia', 'Acre',
                'Roraima', 'Amapá', 'Mato Grosso do Sul', 'Mato Grosso',
                'Maranhão', 'Espírito Santo', 'Goiás', 'Piauí',
                'Outras Unidades da Federação', 'São Paulo', 'Paraná', 'Ceará',
                'Pará', 'Goiás', 'Rondônia', 'Amapá', 'Paraíba', 'Espírito Santo',
                'Maranhão', 'Sergipe'], dtype=object)
```

```
In [19]: STATE_MAPPING = {
    'acre': 'Acre',
    'alagoas': 'Alagoas',
    'amapá': 'Amapá',
    'amazonas': 'Amazonas',
    'bahia': 'Bahia',
    'ceará': 'Ceará',
    'distrito federal': 'Distrito Federal',
    'mato grosso': 'Mato Grosso',
    'mato grosso do sul': 'Mato Grosso do Sul',
    'minas gerais': 'Minas Gerais',
    'outras unidades da federação': 'Outras Unidades da Federação',
    'pará': 'Pará',
    'paraná': 'Paraná',
    'pernambuco': 'Pernambuco',
    'piauí': 'Piauí',
    'rio de janeiro': 'Rio de Janeiro',
    'rio grande do norte': 'Rio Grande do Norte',
    'rio grande do sul': 'Rio Grande do Sul',
    'roraima': 'Roraima',
    'santa catarina': 'Santa Catarina',
    'são paulo': 'São Paulo',
    'sergipe': 'Sergipe',
    'tocantins': 'Tocantins',
    'amapá': 'Amapá',
    'ceará': 'Ceará',
    'espírito santo': 'Espírito Santo',
    'goiás': 'Goiás',
    'maranhão': 'Maranhão',
    'paraíba': 'Paraíba',
```

```

    'pará': 'Pará',
    'paraná': 'Paraná',
    'rondônia': 'Rondônia',
    'são paulo': 'São Paulo',
}

df['state'] = map_strict(df['state'], STATE_MAPPING, 'STATE_MAPPING')

```

```
In [20]: df['state'].unique()
```

```
Out[20]: array(['Bahia', 'São Paulo', 'Paraná', 'Rio Grande do Sul',
                'Rio de Janeiro', 'Pará', 'Ceará', 'Santa Catarina',
                'Distrito Federal', 'Amazonas', 'Rio Grande do Norte', 'Paraíba',
                'Pernambuco', 'Alagoas', 'Minas Gerais', 'Rondônia', 'Acre',
                'Roraima', 'Amapá', 'Mato Grosso do Sul', 'Mato Grosso',
                'Maranhão', 'Espírito Santo', 'Goiás', 'Piauí',
                'Outras Unidades da Federação', 'Sergipe'], dtype=object)
```

```
In [21]: STATE_REGION_MAPPING = {
    'Acre': 'Região Norte',
    'Amapá': 'Região Norte',
    'Amazonas': 'Região Norte',
    'Pará': 'Região Norte',
    'Rondônia': 'Região Norte',
    'Roraima': 'Região Norte',
    'Tocantins': 'Região Norte',
    'Alagoas': 'Região Nordeste',
    'Bahia': 'Região Nordeste',
    'Ceará': 'Região Nordeste',
    'Maranhão': 'Região Nordeste',
    'Paraíba': 'Região Nordeste',
    'Pernambuco': 'Região Nordeste',
    'Piauí': 'Região Nordeste',
    'Rio Grande do Norte': 'Região Nordeste',
    'Sergipe': 'Região Nordeste',
    'Distrito Federal': 'Região Centro-Oeste',
    'Goiás': 'Região Centro-Oeste',
    'Mato Grosso': 'Região Centro-Oeste',
    'Mato Grosso do Sul': 'Região Centro-Oeste',
    'Espírito Santo': 'Região Sudeste',
    'Minas Gerais': 'Região Sudeste',
    'Rio de Janeiro': 'Região Sudeste',
    'São Paulo': 'Região Sudeste',
    'Paraná': 'Região Sul',
    'Rio Grande do Sul': 'Região Sul',
    'Santa Catarina': 'Região Sul',
    'Outras Unidades da Federação': 'Outros',
}

df['state_region'] = map_strict(
    df['state'], STATE_REGION_MAPPING, 'STATE_REGION_MAPPING'
)
df['state_region'].value_counts(dropna=False)
```

```
Out[21]: state_region
Região Sul                282071
Região Norte              206837
Região Nordeste           182973
Região Sudeste            139709
Outros                    94831
Região Centro-Oeste       78853
Name: count, dtype: int64
```

Origin Country field

```
In [22]: df[df['origin_country'].str.contains('✦')]['origin_country'].unique()
```

```
Out[22]: array(['Afeganist✦o', 'Alb✦nia', 'Ant✦gua e Barbuda', 'Arg✦lia',
                'Arm✦nia', 'Ar✦bia Saudita', 'Austr✦lia', 'Azerbaij✦o',
                'Bielor✦ssia', 'Bol✦via', 'Bulg✦ria', 'But✦o', 'Belgica',
                'B✦snia-Herzegovina', 'Camar✦es', 'Canad✦', 'Casaquist✦o',
                'Col✦mbia', 'Cro✦cia', 'Emirados ✦rabes Unidos', 'Eritr✦ia',
                'Eslov✦quia', 'Eslov✦nia', 'Est✦nia', 'Eti✦pia', 'Finl✦ndia',
                'Fran✦a', 'Gab✦o', 'Ge✦rgia', 'Gr✦cia', 'Guin✦',
                'Guin✦ Equatorial', 'Guin✦-Bissau', 'G✦mbia', 'Indon✦sia', 'Ir✦',
                'Isl✦ndia', 'It✦lia', 'I✦men', 'Jap✦o', 'Jord✦nia', 'Let✦nia',
                'Lib✦ria', 'Litu✦nia', 'L✦bano', 'L✦bia', 'Maced✦nia', 'Malau✦',
                'Mal✦sia', 'Maurit✦nia', 'Maur✦cio', 'Mold✦via', 'Mong✦lia',
                'Mo✦ambique', 'M✦xico', 'M✦naco', 'Nam✦bia', 'Nicar✦gua',
                'Nig✦ria', 'Nova Zel✦ndia', 'Om✦', 'Panam✦', 'Papua Nova Guin✦',
                'Paquist✦o', 'Pa✦ses n✦o especificados', 'Pol✦nia', 'Quirguist✦o',
                'Qu✦nia', 'Rep✦blica Centro Africana',
                'Rep✦blica Democr✦tica do Congo', 'Rep✦blica Dominicana',
                'Rep✦blica Tcheca', 'Rep✦blica da Coreia', 'Rep✦blica do Kosovo',
                'Rom✦nia', 'R✦ssia', 'Santa L✦cia', 'Som✦lia', 'Sud✦o', 'Su✦cia',
                'Su✦a', 'S✦o Crist✦v✦o e Nevis', 'S✦o Tome e Pr✦ncipe', 'S✦rvia',
                'S✦ria', 'Tail✦ndia', 'Tanz✦nia', 'Tun✦sia', 'Turcomenist✦o',
                'Ucr✦nia', 'Uzbequist✦o', 'Vietn✦', 'Zimb✦bue', 'Z✦mbia',
                '✦frica do Sul', '✦ustria', '✦ndia', 'Sud✦o do Sul',
                'S✦o Vicente e Granadinas', 'Tadjiquist✦o', 'N✦ger', 'Suazil✦ndia',
                'Ilhas Salom✦o', 'Micron✦sia'], dtype=object)
```

```
In [23]: COUNTRY_MAPPING = {
        'afeganist✦o': 'Afeganist✦o',
        'alemanha': 'Alemanha',
        'andorra': 'Andorra',
        'angola': 'Angola',
        'antilhas holandesas': 'Antilhas Holandesas',
        'ar✦bia saudita': 'Ar✦bia Saudita',
        'argentina': 'Argentina',
        'austr✦lia': 'Austr✦lia',
        'bahamas': 'Bahamas',
        'bangladesh': 'Bangladesh',
        'barbados': 'Barbados',
        'barein': 'Barein',
        'belize': 'Belize',
        'benin': 'Benin',
        'bol✦via': 'Bol✦via',
        'botsuana': 'Botsuana',
```

'brunei': 'Brunei',
'bulgária': 'Bulgária',
'burkina faso': 'Burkina Faso',
'burundi': 'Burundi',
'bélgica': 'Bélgica',
'cabo verde': 'Cabo Verde',
'camboja': 'Camboja',
'canadá': 'Canadá',
'catar': 'Catar',
'chade': 'Chade',
'chile': 'Chile',
'china': 'China',
'china, hong kong': 'China, Hong Kong',
'china, macao': 'China, Macao',
'chipre': 'Chipre',
'cingapura': 'Cingapura',
'colômbia': 'Colômbia',
'comores': 'Comores',
'congo': 'Congo',
'coreia do norte': 'Coreia do Norte',
'costa do marfim': 'Costa do Marfim',
'costa rica': 'Costa Rica',
'croácia': 'Croácia',
'cuba': 'Cuba',
'curaçao': 'Curaçao',
'dinamarca': 'Dinamarca',
'djibouti': 'Djibouti',
'dominica': 'Dominica',
'egito': 'Egito',
'el salvador': 'El Salvador',
'equador': 'Equador',
'eslováquia': 'Eslováquia',
'eslovênia': 'Eslovênia',
'espanha': 'Espanha',
'estados unidos': 'Estados Unidos',
'estônia': 'Estônia',
'fiji': 'Fiji',
'filipinas': 'Filipinas',
'finlândia': 'Finlândia',
'frança': 'França',
'gana': 'Gana',
'granada': 'Granada',
'grécia': 'Grécia',
'guatemala': 'Guatemala',
'guiana': 'Guiana',
'guiana francesa': 'Guiana Francesa',
'haiti': 'Haiti',
'holanda': 'Holanda',
'honduras': 'Honduras',
'hungria': 'Hungria',
'ilhas marshall': 'Ilhas Marshall',
'indonésia': 'Indonésia',
'iraque': 'Iraque',
'irlanda': 'Irlanda',
'irã': 'Irã',
'israel': 'Israel',

'itália': 'Itália',
'jamaica': 'Jamaica',
'japão': 'Japão',
'kiribati': 'Kiribati',
'kuwait': 'Kuwait',
'laos': 'Laos',
'lesoto': 'Lesoto',
'letônia': 'Letônia',
'liechtenstein': 'Liechtenstein',
'lituânia': 'Lituânia',
'luxemburgo': 'Luxemburgo',
'líbano': 'Líbano',
'madagascar': 'Madagascar',
'maldivas': 'Maldivas',
'mali': 'Mali',
'malta': 'Malta',
'malásia': 'Malásia',
'marrocos': 'Marrocos',
'moçambique': 'Moçambique',
'montenegro': 'Montenegro',
'mianma': 'Mianma',
'méxico': 'México',
'nauru': 'Nauru',
'nepal': 'Nepal',
'nicarágua': 'Nicarágua',
'nigéria': 'Nigéria',
'noruega': 'Noruega',
'nova zelândia': 'Nova Zelândia',
'outros países': 'Outros países',
'outros países da américa central e caribe': 'Outros países da América Central',
'outros países da europa': 'Outros países da Europa',
'outros países da oceania': 'Outros países da Oceania',
'outros países da ásia': 'Outros países da Ásia',
'outros países da áfrica': 'Outros países da África',
'palau': 'Palau',
'palestina': 'Palestina',
'panamá': 'Panamá',
'papua nova guiné': 'Papua Nova Guiné',
'paquistão': 'Paquistão',
'paraguai': 'Paraguai',
'países não especificados': 'Países não especificados',
'peru': 'Peru',
'polônia': 'Polônia',
'porto rico': 'Porto Rico',
'portugal': 'Portugal',
'quirguistão': 'Quirguistão',
'quênia': 'Quênia',
'reino unido': 'Reino Unido',
'república da coreia': 'República da Coreia',
'república do kosovo': 'República do Kosovo',
'república dominicana': 'República Dominicana',
'república tcheca': 'República Tcheca',
'romênia': 'Romênia',
'ruanda': 'Ruanda',
'rússia': 'Rússia',
'samoa': 'Samoa',

'san marino': 'San Marino',
'seichelles': 'Seichelles',
'senegal': 'Senegal',
'serra leoa': 'Serra Leoa',
'sri lanka': 'Sri Lanka',
'suriname': 'Suriname',
'suécia': 'Suécia',
'suíça': 'Suíça',
'sérvia': 'Sérvia',
'síria': 'Síria',
'tadjiquistão': 'Tadjiquistão',
'tailândia': 'Tailândia',
'taiwan': 'Taiwan',
'timor leste': 'Timor Leste',
'togo': 'Togo',
'tonga': 'Tonga',
'trinidad e tobago': 'Trinidad e Tobago',
'tunísia': 'Tunísia',
'turquia': 'Turquia',
'tuvalu': 'Tuvalu',
'ucrânia': 'Ucrânia',
'uganda': 'Uganda',
'uruguai': 'Uruguai',
'uzbequistão': 'Uzbequistão',
'vanuatu': 'Vanuatu',
'vaticano': 'Vaticano',
'venezuela': 'Venezuela',
'áfrica do sul': 'África do Sul',
'áustria': 'Áustria',
'índia': 'Índia',
'afeganistão': 'Afeganistão',
'albânia': 'Albânia',
'antígua e barbuda': 'Antígua e Barbuda',
'argélia': 'Argélia',
'armênia': 'Armênia',
'arábia saudita': 'Arábia Saudita',
'austrália': 'Austrália',
'azerbaidjão': 'Azerbaijão',
'bielorrússia': 'Bielorrússia',
'bolívia': 'Bolívia',
'bulgária': 'Bulgária',
'butão': 'Butão',
'bélgica': 'Bélgica',
'bósnia-herzegovina': 'Bósnia-Herzegovina',
'camarões': 'Camarões',
'canadá': 'Canadá',
'casaquistão': 'Casaquistão',
'colômbia': 'Colômbia',
'croácia': 'Croácia',
'emirados árabes unidos': 'Emirados Árabes Unidos',
'eritreia': 'Eritreia',
'eslováquia': 'Eslováquia',
'eslovênia': 'Eslovênia',
'estônia': 'Estônia',
'etiópia': 'Etiópia',
'finlândia': 'Finlândia',

'frança': 'França',
'gabão': 'Gabão',
'geórgia': 'Geórgia',
'grécia': 'Grécia',
'guiné': 'Guiné',
'guiné equatorial': 'Guiné Equatorial',
'guiné-bissau': 'Guiné-Bissau',
'gâmbia': 'Gâmbia',
'ilhas salomão': 'Ilhas Salomão',
'indonésia': 'Indonésia',
'irã': 'Irã',
'islândia': 'Islândia',
'itália': 'Itália',
'iêmen': 'Iêmen',
'japão': 'Japão',
'jordânia': 'Jordânia',
'letônia': 'Letônia',
'libéria': 'Libéria',
'lituânia': 'Lituânia',
'líbano': 'Líbano',
'líbia': 'Líbia',
'macedônia': 'Macedônia',
'malauí': 'Malauí',
'malásia': 'Malásia',
'mauritânia': 'Mauritânia',
'maurício': 'Maurício',
'micronésia': 'Micronésia',
'moldávia': 'Moldávia',
'mongólia': 'Mongólia',
'moçambique': 'Moçambique',
'méxico': 'México',
'mônaco': 'Mônaco',
'namíbia': 'Namíbia',
'nicarágua': 'Nicarágua',
'nigéria': 'Nigéria',
'nova zelândia': 'Nova Zelândia',
'níger': 'Níger',
'omã': 'Omã',
'panamá': 'Panamá',
'papua nova guiné': 'Papua Nova Guiné',
'paquistão': 'Paquistão',
'países não especificados': 'Países não especificados',
'polônia': 'Polônia',
'quirguistão': 'Quirguistão',
'quênia': 'Quênia',
'república centro africana': 'República Centro-Africana',
'república democrática do congo': 'República Democrática do Congo',
'república da coreia': 'República da Coreia',
'república do kosovo': 'República do Kosovo',
'república dominicana': 'República Dominicana',
'república tcheca': 'República Tcheca',
'romênia': 'Romênia',
'rússia': 'Rússia',
'santa lúcia': 'Santa Lúcia',
'somália': 'Somália',
'suazilândia': 'Suazilândia',

```

'sudão': 'Sudão',
'sudão do sul': 'Sudão do Sul',
'suécia': 'Suécia',
'suíça': 'Suíça',
'são cristóvão e nevis': 'São Cristóvão e Nevis',
'são tome e príncipe': 'São Tomé e Príncipe',
'são vicente e granadinas': 'São Vicente e Granadinas',
'sérvia': 'Sérvia',
'síria': 'Síria',
'tadjiquistão': 'Tadjiquistão',
'tailândia': 'Tailândia',
'tanzânia': 'Tanzânia',
'tunísia': 'Tunísia',
'turcomenistão': 'Turcomenistão',
'ucrânia': 'Ucrânia',
'uzbequistão': 'Uzbequistão',
'vietnã': 'Vietnã',
'zimbábue': 'Zimbábue',
'zâmbia': 'Zâmbia',
'áfrica do sul': 'África do Sul',
'índia': 'Índia',
'áustria': 'Áustria',
}

df['origin_country'] = map_strict(
    df['origin_country'], COUNTRY_MAPPING, 'COUNTRY_MAPPING'
)

```

```
In [24]: df[df['origin_country'].str.contains('🇵🇹')]['origin_country'].unique()
```

```
Out[24]: array([], dtype=object)
```

```
In [25]: COUNTRY_CULTURAL_SUBDIVISION_MAPPING = {
    # MENA
    'Afeganistão': 'MENA',
    'Argélia': 'MENA',
    'Arábia Saudita': 'MENA',
    'Barein': 'MENA',
    'Catar': 'MENA',
    'Egito': 'MENA',
    'Emirados Árabes Unidos': 'MENA',
    'Irã': 'MENA',
    'Iraque': 'MENA',
    'Israel': 'MENA',
    'Iêmen': 'MENA',
    'Jordânia': 'MENA',
    'Kuwait': 'MENA',
    'Líbano': 'MENA',
    'Líbia': 'MENA',
    'Marrocos': 'MENA',
    'Mauritânia': 'MENA',
    'Omã': 'MENA',
    'Palestina': 'MENA',
    'Síria': 'MENA',
    'Sudão': 'MENA',

```

```
'Tunísia': 'MENA',
'Turquia': 'MENA',
'Djibouti': 'MENA',
# LATAM
'Antígua e Barbuda': 'LATAM',
'Antilhas Holandesas': 'LATAM',
'Argentina': 'LATAM',
'Bahamas': 'LATAM',
'Barbados': 'LATAM',
'Belize': 'LATAM',
'Bolívia': 'LATAM',
'Chile': 'LATAM',
'Colômbia': 'LATAM',
'Costa Rica': 'LATAM',
'Cuba': 'LATAM',
'Curaçao': 'LATAM',
'Dominica': 'LATAM',
'El Salvador': 'LATAM',
'Equador': 'LATAM',
'Granada': 'LATAM',
'Guatemala': 'LATAM',
'Guiana': 'LATAM',
'Guiana Francesa': 'LATAM',
'Haiti': 'LATAM',
'Honduras': 'LATAM',
'Jamaica': 'LATAM',
'México': 'LATAM',
'Nicarágua': 'LATAM',
'Panamá': 'LATAM',
'Paraguai': 'LATAM',
'Peru': 'LATAM',
'Porto Rico': 'LATAM',
'República Dominicana': 'LATAM',
'Santa Lúcia': 'LATAM',
'São Cristóvão e Nevis': 'LATAM',
'São Vicente e Granadinas': 'LATAM',
'Suriname': 'LATAM',
'Trinidad e Tobago': 'LATAM',
'Uruguai': 'LATAM',
'Venezuela': 'LATAM',
'Outros países da América Central e Caribe': 'LATAM',
# APAC
'Austrália': 'APAC',
'Bangladesh': 'APAC',
'Brunei': 'APAC',
'Butão': 'APAC',
'Camboja': 'APAC',
'Casaquistão': 'APAC',
'China': 'APAC',
'China, Hong Kong': 'APAC',
'China, Macao': 'APAC',
'Cingapura': 'APAC',
'Coreia do Norte': 'APAC',
'Fiji': 'APAC',
'Filipinas': 'APAC',
'Ilhas Marshall': 'APAC',
```

```
'Ilhas Salomão': 'APAC',
'Índia': 'APAC',
'Indonésia': 'APAC',
'Japão': 'APAC',
'Kiribati': 'APAC',
'Laos': 'APAC',
'Malásia': 'APAC',
'Maldivas': 'APAC',
'Micronésia': 'APAC',
'Mianma': 'APAC',
'Mongólia': 'APAC',
'Nauru': 'APAC',
'Nepal': 'APAC',
'Nova Zelândia': 'APAC',
'Palau': 'APAC',
'Papua Nova Guiné': 'APAC',
'Paquistão': 'APAC',
'Quirguistão': 'APAC',
'República da Coreia': 'APAC',
'Samoa': 'APAC',
'Sri Lanka': 'APAC',
'Tadjiquistão': 'APAC',
'Tailândia': 'APAC',
'Taiwan': 'APAC',
'Timor Leste': 'APAC',
'Tonga': 'APAC',
'Turcomenistão': 'APAC',
'Tuvalu': 'APAC',
'Uzbequistão': 'APAC',
'Vanuatu': 'APAC',
'Vietnã': 'APAC',
'Outros países da Oceania': 'APAC',
'Outros países da Ásia': 'APAC',
# Anglo-America / English-Speaking World
'Canadá': 'Anglo-America',
'Estados Unidos': 'Anglo-America',
'Irlanda': 'Anglo-America',
'Reino Unido': 'Anglo-America',
# Nordics / Baltics
'Dinamarca': 'Nordics / Baltics',
'Estônia': 'Nordics / Baltics',
'Finlândia': 'Nordics / Baltics',
'Islândia': 'Nordics / Baltics',
'Letônia': 'Nordics / Baltics',
'Lituânia': 'Nordics / Baltics',
'Noruega': 'Nordics / Baltics',
'Suécia': 'Nordics / Baltics',
# Western / Eastern Europe
'Albânia': 'Western / Eastern Europe',
'Alemanha': 'Western / Eastern Europe',
'Andorra': 'Western / Eastern Europe',
'Armênia': 'Western / Eastern Europe',
'Áustria': 'Western / Eastern Europe',
'Azerbaijão': 'Western / Eastern Europe',
'Bélgica': 'Western / Eastern Europe',
'Bielorrússia': 'Western / Eastern Europe',
```

'Bósnia-Herzegovina': 'Western / Eastern Europe',
'Bulgária': 'Western / Eastern Europe',
'Chipre': 'Western / Eastern Europe',
'Croácia': 'Western / Eastern Europe',
'Eslováquia': 'Western / Eastern Europe',
'Eslovênia': 'Western / Eastern Europe',
'Espanha': 'Western / Eastern Europe',
'França': 'Western / Eastern Europe',
'Geórgia': 'Western / Eastern Europe',
'Grécia': 'Western / Eastern Europe',
'Holanda': 'Western / Eastern Europe',
'Hungria': 'Western / Eastern Europe',
'Itália': 'Western / Eastern Europe',
'Liechtenstein': 'Western / Eastern Europe',
'Luxemburgo': 'Western / Eastern Europe',
'Macedônia': 'Western / Eastern Europe',
'Malta': 'Western / Eastern Europe',
'Moldávia': 'Western / Eastern Europe',
'Mônaco': 'Western / Eastern Europe',
'Montenegro': 'Western / Eastern Europe',
'Polônia': 'Western / Eastern Europe',
'Portugal': 'Western / Eastern Europe',
'República do Kosovo': 'Western / Eastern Europe',
'República Tcheca': 'Western / Eastern Europe',
'Romênia': 'Western / Eastern Europe',
'Rússia': 'Western / Eastern Europe',
'San Marino': 'Western / Eastern Europe',
'Sérvia': 'Western / Eastern Europe',
'Suíça': 'Western / Eastern Europe',
'Ucrânia': 'Western / Eastern Europe',
'Vaticano': 'Western / Eastern Europe',
'Outros países da Europa': 'Western / Eastern Europe',

Sub-Saharan Africa

'África do Sul': 'Sub-Saharan Africa',
'Angola': 'Sub-Saharan Africa',
'Benin': 'Sub-Saharan Africa',
'Botsuana': 'Sub-Saharan Africa',
'Burkina Faso': 'Sub-Saharan Africa',
'Burundi': 'Sub-Saharan Africa',
'Cabo Verde': 'Sub-Saharan Africa',
'Camarões': 'Sub-Saharan Africa',
'Chade': 'Sub-Saharan Africa',
'Comores': 'Sub-Saharan Africa',
'Congo': 'Sub-Saharan Africa',
'Costa do Marfim': 'Sub-Saharan Africa',
'Eritreia': 'Sub-Saharan Africa',
'Etiópia': 'Sub-Saharan Africa',
'Gabão': 'Sub-Saharan Africa',
'Gana': 'Sub-Saharan Africa',
'Guiné': 'Sub-Saharan Africa',
'Guiné Equatorial': 'Sub-Saharan Africa',
'Guiné-Bissau': 'Sub-Saharan Africa',
'Gâmbia': 'Sub-Saharan Africa',
'Lesoto': 'Sub-Saharan Africa',
'Libéria': 'Sub-Saharan Africa',
'Madagascar': 'Sub-Saharan Africa',

```

'Malauí': 'Sub-Saharan Africa',
'Mali': 'Sub-Saharan Africa',
'Maurício': 'Sub-Saharan Africa',
'Moçambique': 'Sub-Saharan Africa',
'Namíbia': 'Sub-Saharan Africa',
'Níger': 'Sub-Saharan Africa',
'Nigéria': 'Sub-Saharan Africa',
'Quênia': 'Sub-Saharan Africa',
'República Centro-Africana': 'Sub-Saharan Africa',
'República Democrática do Congo': 'Sub-Saharan Africa',
'Ruanda': 'Sub-Saharan Africa',
'São Tomé e Príncipe': 'Sub-Saharan Africa',
'Seichelles': 'Sub-Saharan Africa',
'Senegal': 'Sub-Saharan Africa',
'Serra Leoa': 'Sub-Saharan Africa',
'Somália': 'Sub-Saharan Africa',
'Sudão do Sul': 'Sub-Saharan Africa',
'Suazilândia': 'Sub-Saharan Africa',
'Tanzânia': 'Sub-Saharan Africa',
'Togo': 'Sub-Saharan Africa',
'Uganda': 'Sub-Saharan Africa',
'Zimbábue': 'Sub-Saharan Africa',
'Zâmbia': 'Sub-Saharan Africa',
'Outros países da África': 'Sub-Saharan Africa',
# Other / unclassified
'Outros países': 'Outros',
'Países não especificados': 'Outros',
}

df['country_cultural_subdivision'] = map_strict(
    df['origin_country'],
    COUNTRY_CULTURAL_SUBDIVISION_MAPPING,
    'COUNTRY_CULTURAL_SUBDIVISION_MAPPING',
)
df['country_cultural_subdivision'].value_counts(dropna=False)

```

```

Out[25]: country_cultural_subdivision
LATAM                281387
Western / Eastern Europe  231621
APAC                 126753
Outros                88211
Nordics / Baltics     68136
Sub-Saharan Africa    67686
MENA                  65336
Anglo-America         56144
Name: count, dtype: int64

```

Final Validation

```

In [26]: print(f'Null counts:\n{df.isna().sum()}')
print(f'\nUnique countries: {df["origin_country"].nunique()}')
print(f'Date range: {df["date"].min()} to {df["date"].max()}')

assert df.notna().all().all(), 'unexpected nulls after cleaning'

```

```
Null counts:
access_route      0
state             0
origin_country    0
month             0
year              0
arrival_count     0
month_num         0
date              0
year_month        0
state_region      0
country_cultural_subdivision  0
dtype: int64
```

Unique countries: 209

Date range: 1989-01-31 00:00:00 to 2026-08-31 00:00:00

```
In [27]: df.to_parquet(
          os.path.join(ROOT_PATH, 'data', 'processed', 'cleaned_arrivals.parquet'),
          index=False,
          )
```